

Blockchain-based proof of retrievability scheme with data deduplication function for cloud storage

Peiru Yang , Jinyong Chang *, Youtong Wang , Xia Zhou , Yanyan Ji

School of Information and Control Engineering, Xi'an University of Architecture and Technology, 710055, Xi'an, Shaanxi, PR China

ARTICLE INFO

Keywords:

Blockchain
Proof of retrievability
Data deduplication
Cloud storage
Integrity auditing

ABSTRACT

In cloud storage mode, data owner (DO) is allowed to rent storage space of cloud servers to save its local space. But this mode has certain security issues. Due to internal or external factors, cloud servers may lose DO's some data blocks, resulting in economic losses for DOs. Cloud audit scheme is a mechanism that helps DOs detect whether remotely stored data is integral. The current audit schemes still face challenges such as difficulty in integrating with data deduplication mechanisms, high storage overhead for multiple replicas, and privacy leakage during the audit process. In this paper, we will propose a new cloud audit system using technologies such as multi-key server-assisted key generation, minimum bandwidth regeneration code, and Blockchain-assisted transparent auditing. Specifically, using multi-key servers to calculate and generate the same encryption and authentication keys for the same data file to ensure the uniqueness of the stored data. Generate auxiliary information for each data file using the regeneration code to ensure that data blocks can still be recovered once some blocks are lost or damaged. Utilizing Blockchain technology to achieve transparency, traceability, and immutability in the audit process. Finally, the performance analysis shows that our proposed system has certain advantages in communication redundancy, computation costs and storage overheads, while establishing a strong trust model via the blockchain.

1. Introduction

With the rapid development of information and digital technology, individuals and businesses generate a large amount of data, which has brought great pressure to local storage. How to store and manage these massive amounts of data has become a critical issue that they urgently need to address. Cloud storage technology provides the possibility for remote storage of massive amounts of data. Under the cloud storage framework, DO can rent a certain amount of space from cloud service provider (CSP) to store and manage its data. This not only saves local storage space, but also enables convenient access and secure sharing of remote data [1]. Therefore, more and more users are choosing cloud storage services to achieve remote management of their data.

It should be noted that cloud storage technology faces certain security risks [2]. Specifically, after storing data in CSP, DO naturally chooses to delete these data to save its local storage space. This results in DO losing physical control over them. On the one hand, CSP may be vulnerable to external attacks, and on the other hand, in order to provide storage services to more users, CSP may intentionally delete DO's some infrequently used data, resulting in permanent loss of user data.

As a result, two problems naturally arise. The first one is how to conduct efficient integrity audits on data stored remotely, and another one is, once the integrity of remote data is discovered to have been compromised, how should DO recover these damaged data [3]. For the first problem, designing an efficient and stylized integrity audit mechanism is currently a mainstream approach and, in recent years, various data integrity audit schemes with good performances have emerged [4,5]. However, they often introduce third-party auditor (TPA) to assist DO in completing audit work. Studies have shown that TPA may challenge the same data blocks multiple times to determine their contents, resulting in privacy leakage of challenged data. At this point, it is necessary to achieve transparency, traceability, and immutability in the audit process. For the second issue, existing implementation methods often choose to store multiple replicas on multiple cloud servers. Once the integrity of one replica is compromised, there are other replicas that can ensure the recoverability of user data [6]. But this model will result in increased storage costs and wastage of storage resources. Hence, when audit results show that the integrity of stored data has been broken, how to efficiently restore the original data has become a meaningful research question.

* Corresponding author.

E-mail addresses: yperu00513@163.com (P. Yang), changjinyong@xauat.edu.cn (J. Chang), 18093387695@163.com (Y. Wang), 1251723616@qq.com (X. Zhou), yanyan.ji@163.com (Y. Ji).

<https://doi.org/10.1016/j.comnet.2026.112099>

Received 10 May 2025; Received in revised form 7 November 2025; Accepted 8 February 2026

Available online 11 February 2026

1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

For CSP, if multiple DOs store the multiple same data file on it, it will also result in a waste of CSP storage space. At this point, data deduplication technology will also become crucial for CSP [7]. Specifically, when DO uploads its data to CSP, the cloud organizer (CO) can first check if the data has already been uploaded, and if not, proceed with the upload. If other users have already stored the data, there is no need to upload it again, and CO only needs to keep a record. This not only saves the communication overhead of the whole system, but also saves some cost for the subsequent uploaders. Therefore, data deduplication is also essential for cloud storage audit systems. In the process of implementing data deduplication, message-locked encryption, whose key is uniquely determined by the data content, is often used to ensure that the same plaintext generates the same ciphertext [8]. Here, we emphasize that, in message-locked encryption, there exists an attack of brute force searching for keys for low entropy data content. Once the key is exhausted, the stored ciphertext data will be completely cracked. To counter this attack, it is necessary to find a key server to assist DO in generating encryption keys with strong randomness for data file [9]. As a result, with the assistance of key server, how to design and implement a secure cloud audit scheme with data deduplication function is an interesting research problem.

Our Techniques and Contributions. To address the challenges in current cloud audit schemes—such as the difficulty in integrating data recovery with data deduplication mechanisms, the high storage overhead of multiple replicas, and privacy leakage during the auditing process—this paper proposes a novel cloud audit system that synergistically integrates several advanced techniques. The key techniques and their corresponding contributions are summarized as follows:

- **Blockchain-Based Transparent Auditing.** We leverage blockchain smart contracts to automate the challenge-generation and proof-verification processes. The smart contract dynamically generates random challenges based on the current audit timestamp, verifies the aggregated proof returned by the CO, and publishes the audit result immutably on the blockchain. This ensures the entire audit process is transparent, traceable, and resistant to tampering or collusion.

Contribution: We propose a publicly verifiable proof-of-retrievability mechanism that preserves privacy and supports data recovery through redundancy.

- **Data Deduplication Based on Multi-Key Servers.** To enable secure and efficient deduplication, we introduce a collaborative key generation protocol involving multiple key servers (KSs). Each data file's encryption and homomorphic-tag keys are generated with the assistance of KSs, using randomized hashing to prevent eavesdropping. A threshold-based approach ensures that even if some KSs are compromised, the correct keys can still be generated as long as a sufficient number of KSs remain honest [10].

Contribution: We design a key generation process that supports secure deduplication and deterministic authentication, while mitigating single-point failure and key leakage risks.

- **Data Recovery Based on Minimum-Bandwidth Regenerating Code.** We employ regenerating codes to encode the original data blocks with redundancy, distributing both data and encoded blocks across multiple CSPs. In the event of data loss or corruption, the CO can efficiently reconstruct the original data by collecting a sufficient subset of blocks from functioning CSPs [11].

Contribution: We implement an efficient data recovery mechanism that minimizes bandwidth consumption and ensures data retrievability even under partial data loss.

In summary, the main contributions of this work are:

1. A blockchain-based proof-of-retrievability scheme that supports public verification, privacy preservation, and data recovery.
2. A multi-key server-assisted key generation protocol that enables secure deduplication and authentication.

3. An efficient data deduplication mechanism that eliminates redundant uploads and storage, validated through bidirectional proof of ownership.

In this work, we consciously opt for a public blockchain as the trust anchor for our auditing scheme. This choice is driven by our primary objective of achieving maximum decentralization and censorship resistance, which are intrinsic properties of public chains. We explicitly acknowledge that this design introduces transaction fees and confirmation latency. Therefore, our scheme is strategically positioned for low-frequency, high-value data integrity audits, such as those for legal documents, archival records, or critical business data, where the cost of assurance is justified by the immense value and sensitivity of the data.

Related Works. We will discuss the related works from the following aspects.

Data integrity auditing constitutes a technical methodology that allows users to verify the integrity of outsourced data. In 2007, Ateniese et al. [12] proposed the first practical provable data possession (PDP) scheme, which implements remote integrity verification through high-frequency probabilistic sampling without requiring full data retrieval. Concurrently, Juels et al. [13] introduced the proof of retrievability (PoR) scheme, which employs similar probabilistic sampling but innovatively integrates erasure coding to achieve partial recovery of corrupted data blocks. A critical distinction lies in their design objectives: PoR emphasizes data retrievability beyond possession proofs, whereas PDP focuses on existence verification. Building upon these foundations, Shacham et al. [14] developed the compact proof of retrievability (CPoR), demonstrating the necessity of constructing semantically secure auditing mechanisms under the random oracle model. Nevertheless, existing solutions universally rely on the strong assumption of fully trusted TPA, rendering it vulnerable to systemic security breaches if TPA is compromised. For example, the TPA may also collaborate with the CSP to achieve some undisclosed objectives to hide data damage, or work with the data owners to avoid penalties [15]. Additional limitations include insecure protocol designs employing static verification tags, which allow adversaries to infer sensitive data through reverse-engineered audit queries [16].

Fortunately, blockchain technology provides a viable solution for decentralized integrity auditing. Blockchain is decentralized, tamper-proof, consistent, and traceable. Therefore, the information stored on the blockchain is open and transparent. In recent years, more and more researchers have used blockchain to replace TPA. Zhang et al. [17] proposed a blockchain-based public data integrity auditing scheme that generates challenge data blocks for each audit round by extracting random seeds from the latest confirmed blockchain blocks. However, such schemes remain limited to single-cloud scenarios, as their application to multi-cloud environments incurs prohibitive computational and communication overhead due to the lack of batch auditing support. To address the specific requirements of multi-cloud storage, researchers have developed decentralized auditing schemes supporting efficient verification. Yang et al. [15] introduced blockchain into multi-cloud storage auditing, employing a cloud service coordinator to aggregate proofs from multiple CSPs and perform batch verification while blocking collusion pathways between TPA and CSPs.

Secure data deduplication technology can improve cloud storage efficiency while maintaining the confidentiality of outsourced data. Traditional encryption methods have different keys and different ciphertexts for the same file encrypted by different users, thus preventing deduplication. To address this limitation, Douceur et al. [18] introduced convergent encryption (CE), which derives encryption key from the hash value of outsourced data. CE was first formalized by Bellare et al. [19] under the name message lock encryption (MLE). However, the method is susceptible to brute-force attack, therefore, Bellare et al. developed the duplicateless encryption for simple storage (DupLESS) framework [20], incorporating a key server into the key generation process, thereby forcing adversary to conduct online brute-force attack via cloud-server

interactions rather than offline attempts. Subsequently, Halevi et al. [21] proposed the concept of proofs of ownership (PoWs) and implemented a Merkle tree-based scheme to prevent attackers from deceiving CSPs using merely file tags. Following this advancement, numerous PoWs schemes emerged to optimize verification efficiency. While PoWs ensures attacker cannot spoof CSPs possessing complete data via hash value, it fails to enable DOs to verify CSPs' comprehensive data preservation. To bridge this gap, Youn et al. [22] established the bidirectional PoWs, guaranteeing mutual security between DO and CSPs. Traditional secure deduplication research primarily focused on data confidentiality. However, when outsourcing data to CSPs, DO equally prioritizes integrity assurance. Thus, integrating auditing capabilities into secure deduplication systems holds significant practical value. Directly applying prior auditing schemes to deduplication-enabled environment, however, incurs substantial storage overhead from redundant authentication tags for identical files.

To address this, Yuan et al. [23] proposed the first cloud storage framework simultaneously supporting data auditing and deduplication. To optimize CSP storage efficiency, their scheme deduplicates both plaintexts and authentication tags, and requires all DOs to compute and upload tags for identical files, sacrificing local computational efficiency. However, this approach only implements plaintext deduplication and auditing while neglecting data confidentiality. Li et al. [24] designed the first cloud storage system supporting encrypted data auditing and deduplication through a fully trusted proxy server generating authentication tags, though maintaining such a server over insecure public networks incurs prohibitive operational costs. Therefore, Miao et al. [25] developed a blockchain-based scheme for shared data integrity auditing with deduplication capability. Zheng et al. [26] constructed a secure deduplication-efficient data integrity auditing scheme leveraging dynamic cloud storage, which achieves authenticator deduplication and significantly reduces computational overhead in the authenticator generation phase. However, in the above scheme, DO needs to store the key of each file, which increases the storage cost of DO. Therefore, Song et al. [27] proposed a blockchain-based mechanism to reduce the cost of local storage of keys while ensuring the recoverability of keys.

2. Preliminaries

In this section, we present the basic notations and technical background of our scheme.

2.1. Basic notations

Table 1 shows the symbols and their descriptions.

2.2. Bilinear maps

Define G_1 and G_T to be two multiplicative cycle groups of prime number q , where the generating element of G_1 is g . Let $e : G_1 \times G_1 \rightarrow G_T$ be a bilinear mapping which has the following properties:

- **Computability.** For valid $x, y \in G_1$, $e(x, y)$ can be calculated.
- **Bilinearity.** For valid $a, b \in \mathbb{Z}_q^*$, $x, y \in G_1$, all have $e(x^a, y^b) = e(x, y)^{ab}$.
- **Non-Degeneracy.** For given valid $x, y \in G_1$, there exists $e(x, y) \neq 1_{G_T}$.

2.3. Discrete logarithm assumption

For the unknown $a \in \mathbb{Z}_q^*$, given g, g^a as input, the output is a . It is computationally infeasible to solve the Discrete Logarithm (DL) problem in G_1 , for any probabilistic polynomial-time (PPT) algorithm

$$\Pr[A(g, g^a) = a] \leq \text{negligible}.$$

Table 1

Symbols and their corresponding descriptions.

Symbols	Descriptions
DO	Data owner
CSP	Cloud service provider
CO	Cloud organizer
KS	Key server
G_1, G_T	Two multiplicative cyclic groups
g	The generator of G_1
prf_1, prf_2	Pseudorandom functions
H_1, H_2	Hash functions
M	Regenerate matrix used to encode and recover data
$f_1, f_2, \dots, f_n$	Elements in group G_1
m	Number of key servers
(s_k, p_k)	Secret-public share for KS_k
F	Original file to be outsourced
t	Threshold of key servers cryptosystem
K	Encryption key of the file F
(ssk, spk)	Signing key pair
C	Ciphertext of F
CID	Identifier of C
ID	Real and unique identity of DO
R	Redundancy data used to recover C
σ	Authenticator-set of C
τ	Number of cloud service providers
TAB	Data distribution record table
SC_A	Smart contract for auditing
$chal$	Challenge message
$Proof$	Integrity proof
i'	Number of corrupted data blocks

2.4. Pseudorandom function

In modern cryptography, a pseudorandom function (PRF) is a deterministic algorithm that takes a secret key and an input value, producing a fixed-length output. Below are its key characteristics:

- **Output Indistinguishability.** The output of a PRF is similar to that of a random function that no efficient adversary, even with substantial computational power, can tell them apart.
- **Determinism.** PRF is deterministic, meaning it produce the same output every time it is given the same input.

2.5. Smart contract

A smart contract is a computer programme that can be automatically executed on the Blockchain. It is written in codes which run on the Blockchain network, the terms and conditions in the contract are in the form of programme codes and are automatically executed under specific conditions. It has the following characteristics:

- **Decentralisation.** Smart contract runs on a decentralised Blockchain network and are not dependent on any single central authority or server.
- **Immutability.** Once a smart contract is deployed on the Blockchain, its code and functionality cannot be altered unless the contract itself allows for modifications.

2.6. Minimum-bandwidth regenerating code

Minimum-Bandwidth Regenerating (MBR) Code, proposed by Dimakis et al. [28], facilitates the effective recovery with minimal bandwidth in the distributed storage system. This method divides file F into n data blocks, with each CSP storing a portion of the encoded data blocks. The detailed encoding for MBR [11] is as follows:

Given a data file F , the DO divides it into n blocks, then computes

$$\begin{pmatrix} F_1 & F_2 & \dots & F_n \end{pmatrix} M \\ = \begin{pmatrix} F_1 & F_2 & \dots & F_n & R_1 & R_2 & \dots & R_n \end{pmatrix}$$

to obtain the data blocks that need to be stored, which include the original data blocks $F_1, F_2, \dots, F_n$ and the redundant data blocks $R_1, R_2, \dots, R_n$. The construction of M is as follows:

- Select a matrix Ψ such that every submatrix of Ψ is invertible. A suitable choice for Ψ is an $(n \times n)$ Cauchy matrix defined

$$\Psi = (\varphi_{ij})_{n \times n} = \begin{bmatrix} \frac{1}{x_1+y_1} & \frac{1}{x_2+y_1} & \dots & \frac{1}{x_n+y_1} \\ \frac{1}{x_1+y_2} & \frac{1}{x_2+y_2} & \dots & \frac{1}{x_n+y_2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{x_1+y_n} & \frac{1}{x_2+y_n} & \dots & \frac{1}{x_n+y_n} \end{bmatrix}$$

where $x_i, y_j \in \mathbb{Z}_q^*$ are distinct elements and $x_i + y_j \neq 0$ for $i, j \in [1, n]$.

- Select $U = (u_{ij})_{n \times n}$ as an invertible matrix such that

$$U_i U_j^T = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

where $U_i = \{u_{i1}, u_{i2}, \dots, u_{in}\}, i \in [1, n], U' = (U^{-1})^T$.

- Let matrix $V = (v_{ij})_{n \times n}$ satisfies $\lambda V = U' \Psi, 1 - \lambda^2 \neq 0$.
- Thus, the calculation is:

$$M = \begin{pmatrix} E & 0 & \dots & 0 & Q_{11} & Q_{21} & \dots & Q_{n1} \\ 0 & E & \dots & 0 & Q_{12} & Q_{22} & \dots & Q_{n2} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & E & Q_{1n} & Q_{2n} & \dots & Q_{nn} \end{pmatrix},$$

where E represents the unit matrix, $Q_{ij} = V_i^T U_j + \varphi_{ij} E, j \in [1, n]$.

Based on this, it can be known from the construction of M that any $(n \times n)$ non-zero submatrix of M is invertible [11].

2.7. Proof of ownership

Proof of Ownership (PoW) constitutes a fundamental cryptographic primitive in decentralized systems, designed to enable the prover to verifiably demonstrate its legitimate possession or control over a specific data to the verifier, without necessarily revealing the data in its entirety or relying on a trusted central authority.

To achieve secure deduplication while minimizing CO overhead, we employ a bidirectional Proofs of Ownership (PoWs) protocol. CO acts as a trust anchor for deduplication but cannot inherently verify DO₂'s data ownership. PoWs ensures DO₂ genuinely possesses the file matching *CID*, preventing free-riding attacks and CO indeed stores the file, preventing false deduplication claims.

3. System and security models

In this section, we will introduce the system and security models as well as the design goals.

3.1. System model

A graphical description of our system model can be found in Fig. 1. More concretely, it includes five entities: DOs, KSs, CO, CSPs and Blockchain.

- **DOs:** As a user of cloud storage service, DO has a large amount of data that needs to be uploaded to multiple clouds. At the same time, it can delegate Blockchain to perform integrity audit on outsourced data. If the audit result is incomplete, DO has the right to request the CO to recover the corrupted data blocks. For the convenience of introducing the data deduplication function, here, DOs are categorized into initial uploader DO₁ and subsequent uploader DO₂.
- **KSs:** In order to avoid the problem of single point of failure of traditional single key server, multiple KSs are introduced in our system. In particular, they share a secret value, interact with the user, execute a blind signature protocol, and publish blind signatures on the Blockchain to assist DO in generating its encryption and signature keys.

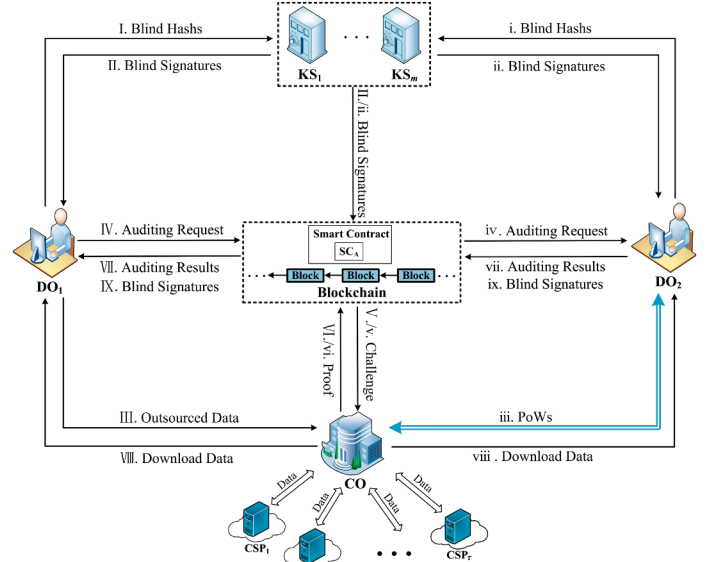


Fig. 1. System model of our scheme.

- **CSPs:** CSP has a huge capacity to provide data storage services for DO. To prevent the loss or damage of stored data caused by unexpected issues in a single cloud, we introduce multiple CSPs to provide more robust data storage services.
- **CO:** The CO is a special CSP that manages the interaction between the DO and the CSPs. It acts as an interface for the CSPs and is responsible for storing information related to the stored data and distributing the data to the appropriate CSPs. Critically, our system design does not assume the CO to be fully trusted. It may attempt to collude with malicious CSPs or behave dishonestly to save resources. To mitigate this, our design leverages Blockchain's transparency to log CO's critical operations, allowing DOs to independently verify CO's behavior. The detailed trust assumption for CO is provided in Section 3.3.
- **Blockchain:** As an entity that is tamper-proof and transparent, Blockchain records the blind signature of KSs on the outsourced data so that the DO can download and recover keys. In addition, Blockchain utilizes smart contract to audit data's integrity based on DO's audit request. Critically, it enhances auditing security and transparency through:
 - **Automated Challenge-Response:** SC dynamically generates random chal using blockchain timestamp (Section 4), eliminating bias risks from manual TPA operations [12,15].
 - **Public Verifiability:** Audit proofs and results are immutably stored on-chain, allowing any entity to independently verify process integrity via published proof.
 - **Collusion Resistance:** Decentralized execution replaces centralized TPA, neutralizing CSP-TPA collusion threats.

Next, we introduce the interaction processes among the above entities, which consists of the following phases: System Initialization, Data Uploading, Integrity Auditing, Data Deduplication, and Data Recovery.

• System Initialization

- $PP \leftarrow \text{SystemSetup}(1^\kappa)$. This is a system initialization algorithm. Given the security parameter κ , this algorithm initializes the system and generates public parameter PP . Without loss of generality, PP is implicitly included in the inputs of other algorithms.
- $(s, \{s_k, p_k\}_{k=1}^m) \leftarrow \text{KeySetup}(m)$. This is a sub-protocol among all the key servers $KS_1, KS_2, \dots, KS_m$. After running this sub-protocol, the outputs include a shared secret value s for the key servers, and the secret-public share (s_k, p_k) for the k -th key server $KS_k, k \in [m]$.

• Data Uploading

- $(K, ssk, spk) \leftarrow \text{KeyGen}(F, \{s_k, p_k\}_{k=1}^m)$. This is a sub-protocol to generate DO's encryption key K and signing key pair (ssk, spk) from data file F . In fact, it requires a certain amount of KSSs to assist in completion, which depends on the threshold size. That is, given the data file F and the shares $\{s_k, p_k\}$, it will finally output K, ssk and spk after completing the interactions with the needed KSSs.
- $(C, CID) \leftarrow \text{Encrypt}(F, K)$. This algorithm is run by DO to generate ciphertext for the data file F based on the encryption key K . Its outputs include the ciphertext C and identifier CID .
- $(R, \sigma) \leftarrow \text{SigGen}(C, CID, ssk)$. This algorithm is executed by the initial uploader DO_1 to encode and sign the ciphertext C . Taking the encrypted file C , identifier CID , and signing key ssk as inputs, this algorithm outputs redundant data R and authenticator-set σ .
- $TAB \leftarrow \text{InitUpload}(CID, C, R, \sigma)$. The algorithm is run by CO to generate storing strategy of DO_1 's data file, which is recorded on a table TAB . That is, after receiving the data files from DO_1 , CO will distribute them to different CSPs according to the strategy recorded in TAB .

• Integrity Auditing

- $chal \leftarrow \text{ChalGen}(CID)$. The algorithm is run by Blockchain to generate a challenge message $chal$, which takes the identifier CID as input, and outputs $chal$.
- $Proof \leftarrow \text{ProofGen}(chal, C, \sigma)$. This algorithm is performed by CO to compute the integrity proof $Proof$. That is, when receiving $chal$ from the BC, the CO interacts with CSPs to generate $Proof$ based on the stored data (C, σ) .
- $1/0 \leftarrow \text{ProofVer}(chal, Proof)$. This is the auditing verification algorithm run by the Blockchain. If the returned $Proof$ is valid, then output 1. Otherwise, output 0.

• Data Deduplication

- $\text{SubUpload}(CID, C)$. This is a sub-protocol between the subsequent uploader DO_2 and CO. Given the ciphertext C that has already been stored by other owner(s), DO and CO need to perform the bidirectional authentication protocol to confirm that both parties truly own the common file C with the identifier CID .

• Data Recovery

- $\{E_k\} \leftarrow \text{ErrorLocate}(C, \sigma)$. If the auditing result of Blockchain is "0", CSPs assist CO to execute this algorithm to determine which data blocks are corrupted. It takes C and σ as inputs, and finally outputs the indexes $\{E_k\}$ of corrupted blocks.
- $C_r \leftarrow \text{DataRecover}(\{E_k\}, C', R')$. This algorithm is executed by CO to recover the corrupted data blocks, which takes $\{E_k\}, C'$ and R' as inputs, and will output the recovered data blocks C_r .

3.2. Security model

In order to achieve integrity auditing, after the DO uploads the data to the CO, any malicious CSP that loses the data cannot generate a forged proof that can be verified by the Blockchain. Moreover, to achieve deduplication, DO_2 and CO need to perform bidirectional verification. We only consider the process in which CO issues a challenge to DO_2 . A malicious DO_2 , in an attempt to gain ownership of the data, falsely claims to CO that it possesses the data.

Based on these two threats, we characterize the security model through the following two games.

Game 1: We suppose a security game 1 against dishonest CSP between a challenger C_1 and a malicious adversary $\mathcal{A}_1$ to describe a security model for storage integrity of cloud data.

1. **Initialization.** C_1 executes **SystemSetup** to generate PP , and sends it to $\mathcal{A}_1$.
2. **Query Phase.** The following queries are allowed for $\mathcal{A}_1$.

- (a) **Hash_query:** The PP includes two hash functions H_1 and H_2 . $\mathcal{A}_1$ can query the hash values of these two hash functions against the values selected adaptively.
- (b) **Private_key_query:** $\mathcal{A}_1$ sends file to C_1 . C_1 performs **KeyGen** to generate keys, then sends them to $\mathcal{A}_1$.
- (c) **Sign_query:** $\mathcal{A}_1$ chooses some data blocks and sends to C_1 . C_1 signs data blocks and returns the signatures to $\mathcal{A}_1$.
3. **Challenge Phase.** C_1 launches a challenge to $\mathcal{A}_1$, who is required to return a proof corresponding to challenge.
4. **Forgery Phase.** $\mathcal{A}_1$ forges proof based on challenge and sends it to C_1 . If it passes verification, $\mathcal{A}_1$ wins.

Definition 1. The scheme is secure against malicious CSP, if the probability of $\mathcal{A}_1$ winning in Game 1 is negligible.

Game 2: We suppose a security game 2 between a challenger C_2 and a malicious adversary $\mathcal{A}_2$ to describe a security model for deduplication.

1. **Initialization.** C_2 executes **SystemSetup** to generate PP , and sends it to $\mathcal{A}_2$.
2. **Query Phase.** $\mathcal{A}_2$ can perform a hash query to C_2 . C_2 returns the result to $\mathcal{A}_2$.
3. **Challenge Phase.** C_2 launches a challenge to $\mathcal{A}_2$, who is required to return a commitment corresponding to challenge.
4. **Forgery Phase.** $\mathcal{A}_2$ forges commitment and sends it to C_2 . If it passes verification, $\mathcal{A}_2$ wins.

Definition 2. The scheme is secure against malicious DO_2 , if the probability of $\mathcal{A}_2$ winning in Game 2 is negligible.

3.3. Trust assumption

This section clarifies the trust assumptions for CO, which is central to our security design. We operate under a "trust-but-verify" paradigm, minimizing reliance on any single entity. We explicitly do not assume a fully trusted CO. It may collude with CSPs or behave dishonestly. However, its ability to harm the system is constrained by the following mechanisms:

1. **Proof Forgery Prevention:** The CO cannot forge audit proofs as it lacks the DO's private key ssk . Proofs are generated from tags stored at CSPs and are directly verified by the Blockchain's smart contract.
2. **Recovery Integrity:** Data recovery relies on publicly verifiable regenerating codes. Providing incorrect blocks during recovery will cause the process to fail, alerting the DO.
3. **Transparency and Accountability:** All critical CO actions (e.g., data distribution, proof aggregation) are logged on the Blockchain, enabling retrospective auditing by DOs.
4. **Deduplication Security:** The bidirectional PoWs prevents the CO from falsely claiming to possess a file it does not have.

In summary, the system's security does not stem from trusting the CO, but from cryptographic protocols and Blockchain transparency that strictly bound the CO's actions, ensuring security guarantees hold even under this relaxed trust assumption.

3.4. Design goals

We construct an integrity auditing scheme with deduplication and data recovery capabilities, which achieves the following design objectives:

- **Functionality.** The scheme we propose covers three basic functionalities: data deduplication, data recovery and integrity auditing. This means it not only enables CO to perform deduplication on ciphertexts, thereby optimizing storage resources and reducing costs, but also equips them with the ability to recover data when data is found to be incomplete or lost, ensuring data security. In addition, it allows

users to conduct integrity auditing of their outsourced data, enabling them to verify the integrity and availability of the data stored in the cloud.

- **Security.** The proposed solution focuses on four key security aspects: non-forgable audit proof, non-forgable data ownership, data confidentiality and privacy preservation, as detailed below:
 - **Non-forgability of audit proof.** If the actual data is lost, the CO cannot forge proof to deceive the Blockchain.
 - **Non-forgability of data ownership.** During the duplicate data upload phase, neither DO₂ nor the CO can pass mutual ownership proof without actual data.
 - **Data confidentiality.** Only DOs possessing the correct decryption keys can access the original information, thereby safeguarding against unauthorized decryption or access by third parties.
 - **Privacy preservation.** During the audit challenge phase, even if multiple DOs have the same data and challenge the same data blocks, the generated proofs will differ.
 - **Security of Key Distribution.** In the *KeyserSetup* process, the coefficients selected by each key server will be broadcasted, which makes the coefficients at risk of being leaked, and may further lead to the theft of the secret value.
- **Efficiency.** Firstly, our scheme effectively reduces the local storage overhead of keys for DOs. Moreover, data encoding and signing are required solely for the DO₁ conducting the initial data upload. Secondly, the scheme leverages the powerful computing capability of the Blockchain, which can verify the integrity of outsourced data with high efficiency. Thirdly, when the audit result indicates incompleteness, CO can promptly and efficiently recover the corrupted data blocks.

4. Our proposed construction

In this section, we focus on the concrete construction of our proposed scheme. The specific details of all phases are as follows.

1. System Initialization

- (a) $PP \leftarrow \text{SystemSetup}(1^\kappa)$: Based on the security parameter κ , this algorithm selects two multiplicative cyclic groups $\mathbb{G}_1$ and $\mathbb{G}_T$. Assume that the generator of $\mathbb{G}_1$ is g , and the orders of $\mathbb{G}_1$ and $\mathbb{G}_T$ equal to q . Define $e: \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_T$ as a bilinear mapping. Then, choose two pseudorandom functions: $prf_1(\cdot): \{1, 2, \dots, n\} \times \mathbb{Z}_q^* \rightarrow \mathbb{Z}_q^*$, $prf_2(\cdot): \mathbb{Z}_q^* \times \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ and two hash functions: $H_1: \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$, $H_2: \mathbb{Z}_q^* \rightarrow \{0, 1\}^\kappa$. Let M be the $(n^2 \times 2n^2)$ -matrix introduced in Section 2, which will be used to recover data. More specifically, the regenerate matrix M has the form

$$M = \begin{pmatrix} E & 0 & \dots & 0 & Q_{11} & Q_{21} & \dots & Q_{n1} \\ 0 & E & \dots & 0 & Q_{12} & Q_{22} & \dots & Q_{n2} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & E & Q_{1n} & Q_{2n} & \dots & Q_{nn} \end{pmatrix},$$

in which E represents the $(n \times n)$ unit matrix and Q_{ij} is an $(n \times n)$ nonsingular matrix. Finally, randomly select n elements $\$1, \$2, \dots, \$n$ from $\mathbb{G}_1$ and publish the public parameter as $PP = \{q, g, \mathbb{G}_1, \mathbb{G}_T, e, prf_1, prf_2, H_1, H_2, M, \$1, \$2, \dots, \$n\}$.

- (b) $(s, \{s_k, p_k\}_{k=1}^m) \leftarrow \text{KeyserSetup}(m)$: KS_k generates a polynomial and broadcasts its coefficients to other servers, sends secret shares to them, and upon receiving shares, verifies their validity. All KS s then aggregate the valid shares, enabling the secret s to achieve threshold sharing through coefficient combination, which requires collaboration from at least t servers for reconstruction. The specific process is as follows:

Define m as the number of key servers. For $k = 1, 2, \dots, m$, each KS_k randomly chooses a polynomial $f_k(x)$ over $\mathbb{Z}_q^*$ with the degree $t - 1$. That is, $f_k(x) = b_{k,0} + b_{k,1}x + \dots + b_{k,t-1}x^{t-1}$. Then, publicly broadcast $\{g^{b_{k,0}}, g^{b_{k,1}}, \dots, g^{b_{k,t-1}}\}$ and secretly send $f_k(l)$ to other

key servers. Based on the received values from other servers, KS_k calculates its secret share $s_k = \sum_{l=1}^m f_l(k)$ and public share $p_k = g^{s_k}$. Finally, the secret value shared by all the key servers is set as $s = \sum_{k=1}^m b_{k,0}$.

2. Data Uploading

- (a) $(K, ssk, spk) \leftarrow \text{KeyGen}(F, \{s_k, p_k\}_{k=1}^m)$: This sub-protocol consists of the following steps.

- DO first divides F into n blocks $f_1, f_2, \dots, f_n$ and randomly chooses $\beta \in \mathbb{Z}_q^*$ as the seed of prf_1 . For $i \in [n + 1]$, generate the randomness $r_i = prf_1(\beta, i)$. Then compute

$$\tilde{f}_i = [H_1(f_i)]^{r_i}, i \in [1, n], \text{ and } \tilde{f}_{n+1} = [H_1(F)]^{r_{n+1}},$$

and send $\{\tilde{f}_i\}_{i=1}^{n+1}$ to all the KS s.

- For $i \in [n + 1]$, the k -th server KS_k computes $x_i^{(k)} = \tilde{f}_i^{s_k}$, and returns $\{x_i^{(k)}\}_{i=1}^{n+1}$ to DO. Meanwhile, it records them on the Blockchain.
- DO verifies whether the Eq. (1) holds.

$$e(\tilde{f}_i, p_k) \stackrel{?}{=} e(x_i^{(k)}, g) \quad (1)$$

If it passes, the value $x_i^{(k)}$ is valid. When t valid values are received, DO can compute the keys K , ssk and spk . Without loss of generality, assume that first t values $x_i^{(1)}, x_i^{(2)}, \dots, x_i^{(t)}$ are valid. DO continues to compute

$$y_k = \prod_{1 \leq \omega \leq t, \omega \neq k} \frac{\omega}{\omega - k}, \quad z_i = \prod_{k=1}^t (x_i^{(k)})^{r_i^{-1} \cdot y_k},$$

for $i \in [n + 1]$. Finally, K , ssk and spk are set as $H_2(\prod_{i=1}^n z_i)$, $H_2(z_{n+1})$, and g^{ssk} , respectively.

- (b) $(C, CID) \leftarrow \text{Encrypt}(F, K)$: DO first chooses any symmetric encryption Enc , such as AES, to encrypt the file F into C . That is, $C = Enc_K(F)$. Then, divide it into

$$C = \{C_1, C_2, \dots, C_n\}, \text{ and } C_i = \{c_{i1}, c_{i2}, \dots, c_{in}\} \in \mathbb{Z}_q^n.$$

Define $CID = H_1(C)$ as the identifier of C .

- (c) $(R, \sigma) \leftarrow \text{SigGen}(C, CID, ssk)$: The initial uploader DO₁, whose unique identity is denoted by ID_1 , performs signature generation as follows. First, compute

$$\begin{pmatrix} C_1 & C_2 & \dots & C_n \end{pmatrix} M = \begin{pmatrix} C_1 & C_2 & \dots & C_n & R_1 & R_2 & \dots & R_n \end{pmatrix},$$

where redundant data $R_i = C_1 Q_{i1} + C_2 Q_{i2} + \dots + C_n Q_{in}$, for $1 \leq i \leq n$. Then, generate the signature σ_i for each data block C_i :

$$\sigma_i = \left[H_1(CID \parallel i) \cdot \prod_{j=1}^n \theta_j^{c_{ij}} \right]^{ssk}.$$

Finally, DO₁ sends ID_1 , CID , C , $R = \{R_1, R_2, \dots, R_n\}$ and $\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_n\}$ to CO.

- (d) $TAB \leftarrow \text{InitUpload}(CID, C, R, \sigma)$: After receiving the messages from DO₁, CO creates a table TAB based on the existing CSP's storage situations to separately store the data file. Denote by θ_k the index set of data blocks to be stored by CSP_k . Record $\{ID_1, CID, \theta_k, CSP_k\}$ on TAB and transmit $ID_1, CID, \{C_i\}, \{R_i\}, \{\sigma_i\}_{i \in \theta_k}$ to CSP_k for storing. Note that $\theta_1, \theta_2, \dots, \theta_\tau$ form a partition of the set $[n]$, where τ is the number of CSPs controlled by this CO.

3. Integrity Auditing

- (a) $chal \leftarrow \text{ChalGen}(CID)$: When Blockchain receives the audit request of CID from DO, it executes smart contract SC_A , which automatically generates number of challenge blocks $l_1 \in [n]$ and randomly chooses a nonempty subset $I \subset [n]$ with $|I| = l_1$. For each $i \in I$, SC_A obtains the microsecond-precision timestamp T and computes the random coefficient $a_i = prf_2(T, i)$ by calling the pseudo-random function prf_2 . Finally, SC_A sends the challenge message $chal = \{CID, \{(i, a_i)\}_{i \in I}\}$ to CO.

- (b) $Proof \leftarrow \mathbf{ProofGen}(chal, C, \sigma)$: After receiving $chal$, CO first looks up the records in TAB to find out the distribution strategy of the data file with the identifier CID , and generates auditing proof as follows. According to the table TAB , CO parses I into $I_1 \cup I_2 \cup \dots \cup I_r$, in which I_k denotes all challenged indexes stored in CSP_k . Then send $chal_k = \{I_k, \{a_i\}_{i \in I_k}, CID\}$ to CSP_k , who calculates

$$\delta_k = \prod_{i \in I_k} \sigma_i^{a_i}, \quad \phi_{kj} = \sum_{i \in I_k} a_i \cdot c_{ij}$$

and sends $Proof_k = \{\delta_k, \{\phi_{kj}\}_{j \in [1, n]}\}$ to CO. Finally, CO aggregates them into

$$\Omega = \prod_{k=1}^r \delta_k = \prod_{i \in I} \sigma_i^{a_i}, \quad \Phi_j = \sum_{k=1}^r \phi_{kj} = \sum_{i \in I} a_i \cdot c_{ij},$$

and records $Proof = \{\Omega, \{\Phi_j\}_{j \in [1, n]}\}$ on the Blockchain as a response to the challenge.

- (c) $1/0 \leftarrow \mathbf{ProofVer}(chal, Proof)$: Upon receiving the returned proof $Proof$ from CO, SC_A verifies the integrity of stored data by checking

$$e(\Omega, g) \stackrel{?}{=} e\left(\prod_{i \in I} H_1(CID \parallel i)^{a_i} \cdot \sum_{j=1}^n \partial_j^{\Phi_j}, spk\right) \quad (2)$$

If it holds, SC_A outputs the auditing result "1". Otherwise, output auditing result "0", which indicates that some data blocks stored on some CSPs are corrupted.

4. Data Deduplication

- (a) $\mathbf{SubUpload}(CID, C)$: If the subsequent uploader DO_2 also has the same data file F , then it can also obtain the same encryption key K from the key servers. Based on this key K , it gets the ciphertext file C and the identifier $CID = H_1(C)$. Then execute a bidirectional PoWs protocol with the CO. The process is shown in Fig. 2 and described as follows.

- DO_2 selects $\xi \in \mathbb{Z}_q^*$ and sends CID, ξ to CO.
- CO checks the existence of C via CID . If CID has not been stored before, CO sends "No-Exist" to DO_2 . Otherwise, CO asks the CSP_k to return the data $\{h_i = H_1(C_i)\}_{i \in \theta_k}$. Upon receiving the data, the CO randomly selects $\zeta \in \mathbb{Z}_q^*$ and sends a commitment

$$\{H_1[h_1 \parallel h_2 \parallel \dots \parallel h_n \parallel \zeta \parallel \xi], \zeta\}$$

to the DO_2 .

- If DO_2 receives "No-Exist", it is allowed to upload the data by executing algorithm $\mathbf{SigGen}$ and stops this algorithm. Otherwise, it verifies the validity of $H_1[h_1 \parallel h_2 \parallel \dots \parallel h_n \parallel \zeta \parallel \xi]$. If the verification passes, DO_2 sends a new commitment $H_1[h_1 \parallel h_2 \parallel \dots \parallel h_n \parallel \xi \parallel \zeta]$ to CO.
- Given the new commitment, CO verifies its validity. If the verification passes, CO sends "Succeed" to DO_2 and stores DO_2 's identity ID_2 in the table TAB , indicating that DO_2 is also the owner of the file with identifier CID . Otherwise, CO sends "No-Exist" to DO_2 .

5. Data Recovery

- (a) $\{E_k\} \leftarrow \mathbf{ErrorLocate}(C, \sigma)$: Upon receiving an auditing result "0" from Blockchain, CO asks CSPs to assist in locating the error(s), which consists of the following steps.

- For $k \in [1, \tau]$, CSP_k initializes an empty set E_k to record the index(es) of erroneous block(s). For each data block C_i ($i \in \theta_k$), it verifies

$$e\left(H_1(CID \parallel i) \cdot \prod_{j=1}^n \partial_j^{c_{ij}}, spk\right) \stackrel{?}{=} e(\sigma_i, g). \quad (3)$$

If it does not hold, then its index will be recorded into E_k . At the end of this step, CSP_k sends E_k to CO.

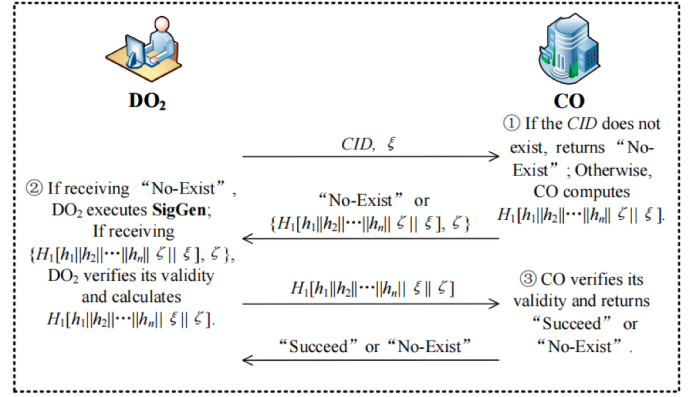


Fig. 2. Procedure of subupload.

- (b) $C_r \leftarrow \mathbf{DataRecover}(\{E_k\}, C', R')$: This algorithm recovers corrupted data blocks based on the uncorrupted files, which consists of the following steps.

- CO generates $E = E_1 \cup E_2 \cup \dots \cup E_\tau$ and checks whether the number of corrupted data blocks $|E|$ is larger than $\lfloor \frac{n}{2} \rfloor$. If it is, then the original data cannot be recovered and record this event on the Blockchain. Otherwise, the corrupted blocks can be recovered as follows. Without loss of generality, assume that the first t' ($1 \leq t' \leq \lfloor \frac{n}{2} \rfloor$) data blocks $C_1, C_2, \dots, C_{t'}$ were corrupted. Then CO collects the remaining $C' = \{C_{t'+1}, C_{t'+2}, \dots, C_n\}$ and the redundant data $R' = \{R_{n-t'+1}, R_{n-t'+2}, \dots, R_n\}$ from the CSPs.
- Since any $n \times n$ non-zero submatrix of M is nonsingular, CO can recover $C_r = \{C_1, C_2, \dots, C_{t'}\}$ by calculating

$$\begin{pmatrix} R_{n-t'+1} & R_{n-t'+n} & \dots & R_n & C_{t'+1} & \dots & C_n \end{pmatrix} \times \begin{pmatrix} Q_{n-t'+1,1} & Q_{n-t'+2,1} & \dots & Q_{n,1} & 0 & \dots & 0 \\ Q_{n-t'+1,2} & Q_{n-t'+2,2} & \dots & Q_{n,2} & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ Q_{n-t'+1,n} & Q_{n-t'+2,n} & \dots & Q_{n,n} & 0 & \dots & E \end{pmatrix}^{-1} = (C_1 \quad C_2 \quad \dots \quad C_{t'} \quad C_{t'+1} \quad \dots \quad C_n) \quad (4)$$

5. Correctness and security analysis

In this section, we analyze the correctness and security of our scheme.

5.1. Correctness analysis

- (1) **Audit Effectiveness.** The correctness of the integrity verification Eq. (2) is determined by the following proof steps:

$$\begin{aligned} e(\Omega, g) &= e\left(\prod_{i \in I} \sigma_i^{a_i}, g\right) \\ &= e\left(\prod_{i \in I} [H_1(CID \parallel i)]^{a_i} \cdot \prod_{j=1}^n \partial_j^{c_{ij} \cdot \sum_{i \in I} a_i}, g\right) \\ &= e\left(\prod_{i \in I} H_1(CID \parallel i)^{a_i} \cdot \prod_{i \in I} \prod_{j=1}^n \partial_j^{a_i \cdot c_{ij}}, g^{ssk}\right) \\ &= e\left(\prod_{i \in I} H_1(CID \parallel i)^{a_i} \cdot \prod_{j=1}^n \partial_j^{\sum_{i \in I} a_i \cdot c_{ij}}, spk\right) \\ &= e\left(\prod_{i \in I} H_1(CID \parallel i)^{a_i} \cdot \prod_{j=1}^n \partial_j^{\Phi_j}, spk\right) \end{aligned}$$

Therefore, we observe that the equation $e(\Omega, g) = e(\prod_{i \in I} H_1(CID \parallel i)^{a_i} \cdot \prod_{j=1}^n \partial_j^{\Phi_j}, spk)$ holds.

- (2) **Retrievability of Erroneous Data Blocks.** The correctness of the Eq. (3) is established through the following proof steps:

$$\begin{aligned} & e\left(H_1(CID \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}}, spk\right) \\ &= e\left(H_1(CID \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}}, g^{ssk}\right) \\ &= e\left([H_1(CID \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}}]^{ssk}, g\right) \\ &= e(\sigma_i, g) \end{aligned}$$

Thus, the equation $e\left(H_1(CID \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}}, spk\right) = e(\sigma_i, g)$ holds.

- (3) **Data Recoverability.** During the data recovery phase, if the number of damaged data blocks does not exceed $\lfloor \frac{n}{2} \rfloor$, the CO can restore the original data C using Eq. (4). Its correctness is proved as follows:

First, we multiply both sides of the Eq. (4) by matrix

$$X = \begin{pmatrix} Q_{n-t'+1,1} & Q_{n-t'+2,1} & \cdots & Q_{n,1} & 0 & \cdots & 0 \\ Q_{n-t'+1,2} & Q_{n-t'+2,2} & \cdots & Q_{n,2} & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ Q_{n-t'+1,n} & Q_{n-t'+2,n} & \cdots & Q_{n,n} & 0 & \cdots & E \end{pmatrix}.$$

Thereby yields (for the sake of convenience, the matrix following the second equality sign is presented in its transpose form (denoted by T))

$$\begin{aligned} & (R_{n-t'+1} \quad R_{n-t'+2} \quad \cdots \quad R_n \quad C_{t'+1} \quad \cdots \quad C_n) \\ &= (C_1 \quad C_2 \quad \cdots \quad C_{t'} \quad C_{t'+1} \quad \cdots \quad C_n) X \\ &= \begin{pmatrix} [C_1 Q_{n-t'+1,1} + C_2 Q_{n-t'+1,2} + \cdots + C_{t'} Q_{n-t'+1,t'}]^T \\ [C_1 Q_{n-t'+2,1} + C_2 Q_{n-t'+2,2} + \cdots + C_{t'} Q_{n-t'+2,t'}]^T \\ \vdots \\ [C_1 Q_{n-t'+n,1} + C_2 Q_{n-t'+n,2} + \cdots + C_{t'} Q_{n-t'+n,t'}]^T \\ C_{t'+1}^T \\ \vdots \\ C_n^T \end{pmatrix}. \end{aligned}$$

For any $i \in [t']$, $R_{n-t'+i} = C_1 Q_{n-t'+i,1} + C_2 Q_{n-t'+i,2} + \cdots + C_{t'} Q_{n-t'+i,t'}$.

Thus, when the number of damaged data blocks does not exceed $\lfloor \frac{n}{2} \rfloor$, the CO can restore the intact data of corrupted blocks.

5.2. Security analysis

In this subsection, we will analyze the security of our scheme from the aspects of auditing authenticity and effectiveness of deduplication verification.

Theorem 1. Under the assumption that the DL problem is computationally hard, the proposed scheme guarantees that a CSP_k cannot feasibly forge valid proofs capable of passing integrity audits when challenged data blocks are incomplete.

Proof. If adversary A_1 wins Game 1 with non-negligible probability, we can construct a simulator C_1 that solves the hard problem. The simulation proceeds as follows:

1. **Initialization.** For each $1 \leq i \leq n$, C_1 randomly selects $\alpha_i, \beta_i \in Z_q^*$ and sets $\hat{f}_i = g^{\alpha_i} (g^{\beta_i})^{\beta_i}$. C_1 generates the system parameter C_1 executes **SystemSetup**(1^k) to generate $PP = \{q, g, \mathbb{G}_1, \mathbb{G}_T, e, prf_1, prf_2, H_1, H_2, M, \hat{f}_1, \hat{f}_2, \dots, \hat{f}_n\}$, and sends PP to A_1 .
2. **Query Phase.** The following queries are allowed for A_1 .
 - (a) **Hash_query:** A_1 can query the hash values of hash functions against the values selected adaptively. C_1 maintains a list $L = \{H_1(f_i), f_i\}$ locally to query the results. If selected f_i is in L , C_1 returns $H_1(f_i)$ to A_1 . Otherwise, C_1 selects a value $h \in Z_q^*$, and sets $H_1(f_i) = g^h$. C_1 stores new tuple $(f_i, H_1(f_i))$ into L .

- (b) **Private_key_query:** C_1 can perform **KeyGen** to generate keys using an selected file F . C_1 first performs **Hash_query** on the selected f_i . After retrieving the corresponding tuple $(H_1(f_i), f_i)$ in L , C_1 computes $ssk = H_2(\prod_{i=1}^n z_i)$ and returns ssk to A_1 .
- (c) **Sign_query:** C_1 signs data blocks and returns the signatures to A_1 . A_1 first selects $CID = H_1(C)$. For each $1 \leq i \leq n$, C_1 can calculate and return the signatures $\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_n\}$, where

$$\sigma_i = \left[H_1(CID \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}} \right]^{ssk}.$$

3. **Challenge Phase.** C_1 launches a challenge $chal = \{CID, \{(i, a_i)\}_{i \in I}\}$ to A_1 , who is required to return a proof $Proof = \{\Omega, \{\Phi_j\}_{j \in [1,n]}\}$ corresponding to challenge.
4. **Forgery Phase.** C_1 launches challenge $chal$ to A_1 , who is required to return a proof $Proof$ corresponding to challenge. A_1 might want to forge a positive proof $Proof^*$. It will generate verification tags $\{\sigma_i^*\}$ for forged file using the wrong private key ssk^* , where

$$\sigma_i^* = [H_1(CID^* \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}^*}]^{ssk^*},$$

and

$$C^* = \{C_1^*, C_2^*, \dots, C_n^*\}, \quad C_i^* = \{c_{i1}^*, c_{i2}^*, \dots, c_{in}^*\}.$$

Subsequently, it generates the forged certificate

$$Proof_k^* = \{\delta_k^*, \{\phi_{kj}^*\}_{j \in [1,n]}\},$$

where

$$\delta_k^* = \prod_{i \in I_k} \sigma_i^{*a_i}, \quad \phi_{kj}^* = \sum_{i \in I_k} a_i \cdot c_{ij}^*.$$

Finally, A_1 aggregates

$$\Omega^* = \prod_{k=1}^{\tau} \delta_k^* = \prod_{i \in I} \sigma_i^{*a_i}, \quad \Phi_j^* = \sum_{k=1}^{\tau} \phi_{kj}^* = \sum_{i \in I} a_i \cdot c_{ij}^*,$$

and publishes $Proof^* = \{\Omega^*, \{\Phi_j^*\}_{j \in [1,n]}\}$ to the C_1 as a response to the challenge. The verification process is as follows:

$$\begin{aligned} e(\Omega^*, g) &= e\left(\prod_{i \in I} \sigma_i^{*a_i}, g\right) \\ &= e\left(\prod_{i \in I} [H_1(CID \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}^*}]^{ssk^* \cdot a_i}, g\right) \\ &= e\left(\prod_{i \in I} H_1(CID \parallel i)^{a_i} \cdot \prod_{i \in I} \prod_{j=1}^n \vartheta_j^{a_i \cdot c_{ij}^*}, g^{ssk^*}\right) \\ &= e\left(\prod_{i \in I} H_1(CID \parallel i)^{a_i} \cdot \prod_{j=1}^n \vartheta_j^{\sum_{i \in I} a_i \cdot c_{ij}^*}, spk^*\right) \\ &= e\left(\prod_{i \in I} H_1(CID \parallel i)^{a_i} \cdot \prod_{j=1}^n \vartheta_j^{\Phi_j^*}, spk^*\right) \end{aligned}$$

Because ssk^* is not equal to ssk , we can obtain

$$[H_1(CID \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}^*}]^{ssk^*} \neq \sigma_i.$$

Therefore, We can obtain

$$\begin{aligned} & e\left(\prod_{i \in I} [H_1(CID \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}^*}]^{ssk^* \cdot a_i}, g\right) \\ & \neq e(\Omega, g) \end{aligned}$$

If the A_1 forges the auditing certificate, it cannot pass the integrity verification of the C_1 . This also ends the proof of **Theorem 1**.

Theorem 2. During the duplicate data upload phase, malicious user may falsely claim that they possess data already stored in the cloud and attempt to pass the CO's verification. Without genuine data, the DO cannot pass the CO's data ownership verification.

Proof. PoWs directly addresses Game 2 in our threat model (Section 3.2). By requiring bidirectional nonce-based commitments, it ensures a malicious DO₂ without actual data cannot forge commitment and CO cannot deceive DO₂ by falsifying commitment. If the DO₂ or CO does not possess file F , it cannot return the correct verification result.

1. **Initialization.** C_2 runs **SystemSetup** to generate $PP = \{q, g, G_1, G_T, e, \{\$1, \$2, \dots, \$n\}, prf_1, prf_2, H_1, H_2, M\}$, and sends it to A_2 .
2. **Query Phase.** A_2 selects $\xi^* \in \mathbb{Z}_q^*$, sends ξ^* to C_2 . C_2 randomly selects $\zeta^* \in \mathbb{Z}_q^*$ and computes $\{h_i^* = H_1(C_i^*)\}$, then sends $\{H_1[h_1^* || h_2^* || \dots || h_n^* || \zeta^* || \xi^*], \zeta^*\}$ to A_2 .
3. **Challenge Phase.** C_2 randomly selects $\zeta \in \mathbb{Z}_q^*$ and launches a challenge CID and ζ to A_2 , who is required to return a commitment $\{H_1[h_1 || h_2 || \dots || h_n || \zeta || \xi], \zeta\}$ corresponding to CID . The A_2 cannot query CID in query phase.
4. **Forgery Phase.** A_2 forges commitment $H_1[h_1 || h_2 || \dots || h_n || \xi || \zeta]$ and sends it to C_2 . If it passes verification, A_2 wins.

If the DO₂ forges the data ownership verification, it cannot pass the verification of the CO. This also ends the proof of Theorem 2.

Remark 1. Data confidentiality. In the **KeyGen** algorithm, the DO randomizes the data before transmission to the KSs, ensuring that the keys generated do not disclose any information about the plaintext. Subsequently, DO employs this key K to encrypt data by computing $C = Enc_K(F)$. As a result, during both the data uploading and recovery phases, the data is stored and transmitted in ciphertext form. This approach guarantees that only DOs in possession of the correct decryption key can decrypt the ciphertext and retrieve the original plaintext, thereby ensuring the confidentiality of the data.

Remark 2. Privacy preservation. In the **ChalGen** algorithm, the Blockchain incorporates the current time T to generate coefficients $a_i = prf_2(T, i)$, $i \in I$. This ensures that even when challenges are issued for the same block at different times, the generated proofs will be distinct. This protects users' data ownership privacy, prevents potential identity leakage, and precludes inferring user behavior through proof analysis.

Remark 3. Security of Key Distribution. The **KeySetup** protocol ensures robust key distribution security through three core cryptographic principles: Threshold-based secret sharing leverages $(t-1)$ -degree polynomials requiring collusion of at least t malicious KSs to reconstruct the master secret s . Verifiable transmission is achieved by broadcasting polynomial coefficients as $\{g^{b_{k,i}}\}$ rather than raw values, enabling any party to validate secret shares s_k via pairing checks $e(\tilde{f}_i, p_k) \stackrel{?}{=} e(x_i^{(k)}, g)$ during **KeyGen**. Adaptive adversary resistance arises from the requirement that KSs must secretly exchange evaluations $f_k(I)$ over authenticated channels, preventing offline brute-force attacks while ensuring compromise of up to $t-1$ KSs reveals zero information about s . This construction simultaneously mitigates single-point failures, eavesdropping risks, and insider threats.

6. Performances analysis

We primarily conduct comparisons from three aspects: functional comparison, theoretical analysis and simulation experiments.

The Blockchain architecture we explore in this paper is based on public chains like Ethereum, which provides unparalleled decentralization and trust minimization at the cost of transaction fees (Gas) and latency. Acknowledging this trade-off is crucial. The following performance analysis first compares the cryptographic overhead of our scheme with that of other schemes under a defined model. Then, a quantitative cost analysis of on-chain operations is conducted. The symbols used throughout this section are defined in Table 2.

Table 2

Notations For theoretical analysis.

Notations	Descriptions
$ C $	Size of the ciphertext
$ \mathbb{Z}_q^* , q , G_1 , G_T $	The length of elements in $\mathbb{Z}_q^*, q, G_1, G_T$
H, H_2	Size of Two Hash functions
$\mathcal{E}_1, \mathcal{E}_2$	Exponential on G_1, G_T
$\mathcal{M}_1, \mathcal{M}_2$	Multiplication on G_1, G_T
$\mathcal{P}$	A bilinear pairing
$ TB_{name} , TAB $	Length of the table
$ tag , CID $	File tag size
$ sk $	Size of the private key
-	Inapplicability

6.1. Function comparison

In this subsection, we mainly conduct a comparison of the functions of six solutions, and the comparison results are in Table 3. In the functional comparison, we select six literatures relevant to this paper and compared the functional differences implemented in their schemes.

From the comparison results, we find that in Song et al. [27] and Zheng et al. [26] schemes, they only consider data deduplication and do not consider the data recovery process. However, in Qiao et al. [30] and Zhao et al. [9] schemes, they only consider data recovery but do not consider data deduplication. The existing schemes do not consider the simultaneous implementation of both functions. Based on this, our scheme has considered these functions and implemented them, becoming the first Blockchain-based integrity auditing scheme that simultaneously considers data deduplication and data recovery. It fills the gaps of other schemes and has better implementation applicability and practicality.

6.2. Communication overhead

In this subsection, we mainly analyze the communication overhead of each scheme, comparing them from three aspects: key generation, data upload and audit phases. The main processes to consider are: DO $\rightarrow$ CO, DO $\rightarrow$ Blockchain, DO $\rightarrow$ KSs, $CSP_k \rightarrow$ CO, CO $\rightarrow$ Blockchain.

During the key generation phase, DO interacts with KSs, sending

$$\tilde{f}_i = [H_1(f_i)]^{r_i}, i \in [1, n], \text{ and } \tilde{f}_{n+1} = [H_1(F)]^{r_{n+1}},$$

which is of size $(n+1)|H|$.

During the data upload phase, DO interacts with CO, sending the encrypted file C , data block signatures σ and data CID , and R related to data recovery to CO, with a size of $3n(|q| + |G_1|) + |H|$.

In the audit process, CO needs to interact with CSP_k and Blockchain. In this process, we mainly consider the transmission of the proof generated by CO. It needs to retrieve the data and signatures of the challenged blocks from the CSP_k , and finally generates signature aggregation and data aggregation based on these data blocks and signatures, and sends them to the Blockchain. The communication overhead in this process is $k(2|q| + |G_1|)$.

We use the same method to analyze the communication overhead of each process in the other schemes. Their comparison results are shown in Table 4.

From the comparison results, we can see that in the key generation phase, our scheme is more advantageous than scheme [27]. However, in the data upload phase, since our scheme considers data recovery functionality, it adds some necessary information during the data upload process, resulting in greater communication overhead, but it does not affect our efficiency. Overall, our scheme has advantages to some extent.

6.3. Computational cost

In this section, we mainly analyze the computational overhead of our scheme during the upload and audit phases, and compare it with

Table 3
The comparison on the function.

Scheme	Data Recovery	Data Deduplication	Multi-Cloud	Multi-Key Servers	Blockchain-Based	Integrity Auditing
Scheme [29]	×	×	✓	×	×	✓
Scheme [27]	×	✓	×	✓	✓	✓
Scheme [30]	✓	×	✓	×	✓	✓
Scheme [9]	✓	×	✓	×	×	✓
Scheme [26]	×	✓	×	×	×	✓
Our scheme	✓	✓	✓	✓	✓	✓

Table 4
The comparison on the communication cost.

Scheme	DO → CO	DO → Blockchain	DO → KSs	CSP _k → CO	CO → Blockchain
Scheme [27]	$ C + (n+2) G_1 + 2 Z_q^* $	–	$(n+1)H + Z_q^* $	–	–
Scheme [9]	$2(n q + (n+1) G_1)$	–	–	$2l q + (l+1) G_1 $	–
Our scheme	$3n(q + G_1) + H$	$2 G_1 $	$(n+1)H$	$k(2 q + G_1)$	$ q + G_1 $

Table 5
The comparison on the computational cost.

Scheme	InitUpload	ProofGen	ProofVer
Scheme [27]	$n(H_1 + m\mathcal{E}_1 + (m+1)\mathcal{M}_1)$	$l_2(2\mathcal{E}_1 + \mathcal{M}_1 + H_2) + d\mathcal{M}_1 + (l_2 - 1)\mathcal{M}_1$	$2\mathcal{P} + (2 + l_2(l_2 - 1))\mathcal{M}_1 + \mathcal{E}_1 + l_2\pi$
Scheme [9]	$na(2H + H_2 + \mathcal{M}_1 + 2\mathcal{E}_1)$	$2C_2\pi + C_2^2(\mathcal{M}_1 + 2\mathcal{E}_1 + 2(\tau - 1)\mathcal{E}_1 + (\tau - 1)\mathcal{A}_q)$	$2\mathcal{P} + 3(\mathcal{E}_1 + \pi) + (C_2^2 + 1)\mathcal{M}_1 + H_2$
Our Scheme	$n(H_1 + (n+1)(\mathcal{M}_1 + \mathcal{E}_1))$	$2(l+1)\mathcal{E}_1 + lH_1 + (2(\tau + l) - 3)\mathcal{M}_1 q $	$2\mathcal{P} + 2\mathcal{E}_1 + l\mathcal{M}_1$

Table 6
The comparison on the storage cost.

Scheme	DO	CO
Scheme [27]	$ G_1 + 2H + tag + \theta $	–
Scheme [9]	$ sk + tag + 2 Z_q^* $	$ TB_{name} $
Our Scheme	$ Z_q^* + CID $	$ TAB $

other schemes. We have omitted some processes with relatively small computational overhead. The analysis of the computational overhead during the initial data upload and the audit process is as follows.

First, when initially uploading data, DO needs to generate signatures for the n data blocks in the ciphertext, and their calculation process is

$$\sigma_i = [H_1(CID \parallel i) \cdot \prod_{j=1}^n \vartheta_j^{c_{ij}}]^{ssk}, i \in [1, n].$$

The computational overhead for generating a signature for a data block is

$$H_1 + (n+1)(\mathcal{M}_1 + \mathcal{E}_1),$$

so the computational overhead during the signature generation process is

$$n(H_1 + (n+1)(\mathcal{M}_1 + \mathcal{E}_1)).$$

Secondly, during the process of generating the integrity proof, the CO reads the challenge message *chal* on the Blockchain, and executes the proof generation calculation process as follows:

$$\delta_k = \prod_{i \in I_k} \sigma_i^{a_i}, \quad \phi_{kj} = \sum_{i \in I_k} a_i \cdot c_{ij}$$

and

$$\Omega = \prod_{k=1}^{\tau} \delta_k = \prod_{i \in I} \sigma_i^{a_i}, \quad \Phi_j = \sum_{k=1}^{\tau} \phi_{kj} = \sum_{i \in I} a_i \cdot c_{ij}.$$

In the generation of *ProofGen* phase, the computational overhead for CSP_k is

$$2(l+1)\mathcal{E}_1 + lH_1 + 2(l-1)\mathcal{M}_1,$$

and the computational overhead for CO is

$$(2\tau - 1)\mathcal{M}_1.$$

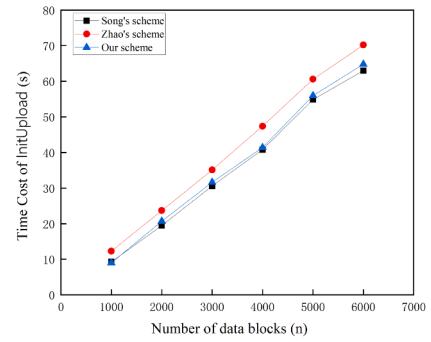


Fig. 3. Time for the generation of signatures.

The computational overhead during proof generation is

$$2(l+1)\mathcal{E}_1 + lH_1 + (2(\tau + l) - 3)\mathcal{M}_1.$$

Finally, during the process of verifying the audit proof, Blockchain needs to verify the audit proof returned by CO, and the judgment process is:

$$e(\Omega, g) = e(\prod_{i \in I} H_1(CID \parallel i)^{a_i} \cdot \sum_{j=1}^n \vartheta_j^{\Phi_j}, spk).$$

The computational overhead of this process is

$$2\mathcal{P} + 2\mathcal{E}_1 + l\mathcal{M}_1.$$

We conduct a computational cost analysis of other related schemes in the same manner, estimated the cost of each entity, and then present the comparison results of total computational cost in Table 5.

From the comparison results, we can see that our scheme, considering the data recovery function, does not have significantly higher computational overhead compared to other schemes. In comparison, during the initial data upload phase, our scheme has lower computational overhead than scheme [27]. In the audit proof verification phase, our scheme has lower computational overhead than scheme [27] and scheme [9]. Overall, our scheme has advantages in terms of computational overhead.

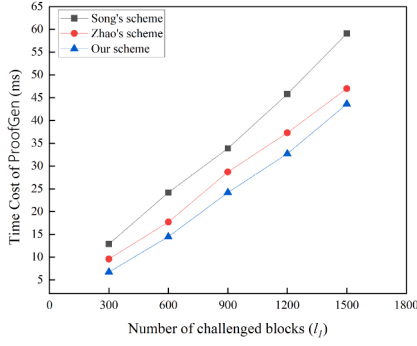


Fig. 4. Time for the generation of proof.

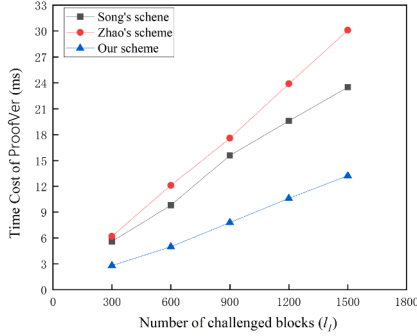


Fig. 5. Time for verification of proof.

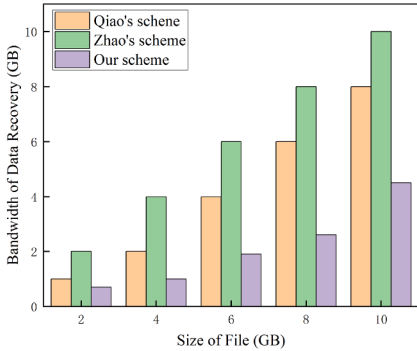


Fig. 6. Bandwidth for the data recovery.

6.4. Storage cost

Here, we primarily compare the storage overhead of the DO and CO entities across different schemes, since the storage cost for all other entities remain identical. For instance, in the CSP, the storage overhead primarily consists of the file and its corresponding signature.

In our scheme, the DO deletes local file after uploading it to the CO. Therefore, the CO only needs to store filename CID and seed $\beta \in \mathbb{Z}_q^*$ generated during data uploading process, avoiding excessive redundant data storage.

The CO distributes the received file requiring storage and other information to the CSP, maintaining only a file registry TAB .

Similarly, we obtain storage overhead for DO and CO in other schemes and present comparison results in Table 6. To achieve deduplication, other schemes incur additional key storage overhead proportional to the number of data blocks, whereas our approach requires storing only a single key seed, thereby reducing client-side storage costs.

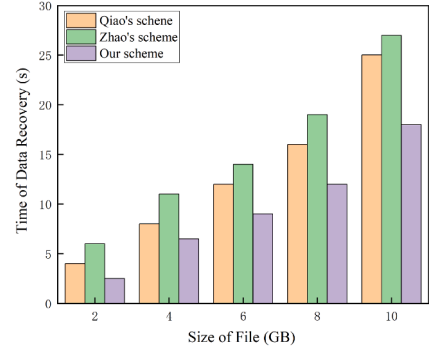


Fig. 7. Time for the data recovery.

6.5. Experimental results

In order to conduct a more effective performance comparison, we conduct experiments to simulate our scheme. We use the Java-based Pairing-Based Cryptography (PBC) library, which is a powerful tool for implementing encryption algorithms based on bilinear pairings. The development tool is IntelliJ IDEA, a widely used Java integrated development environment (IDE) that provides a robust and efficient coding environment. The computer used for the experiment is a Lenovo running Windows 10, equipped with an Intel Core i5-8265U processor, with a base frequency of 1.60GHz (boostable to 1.80GHz). This setup ensures that the experiment is conducted in a realistic and representative computing environment.

To ensure the reliability and comparability of experimental results, we carefully design the following parameters based on cryptographic standards and related works. The prime order q of multiplicative cyclic groups G_1 and G_T is set to 160 bits, which meets the computational hardness requirement of the DL problem in G_1 . This parameter ensures that adversaries cannot solve the DLP within polynomial time, aligning with the security assumptions in Section 5.2. The group size is set to 512 bits to balance security and computational efficiency. This configuration is consistent with the standard in pairing-based cryptography (e.g., [28]), where 512-bit groups provide sufficient resistance against quantum attacks while maintaining feasible computation for bilinear maps $e : G_1 \times G_1 \rightarrow G_T$. Implemented using SHA-256 (256-bit output), which is a widely adopted cryptographic hash function resistant to collision and preimage attacks. The 256-bit output length ensures the uniqueness of data identifiers (e.g., CID) and prevents forgery in deduplication verification (Section IV.4). Employed AES-256, a 256-bit key advanced encryption standard, to ensure data confidentiality. This aligns with the key generation process in Section IV.2.a, where the encryption key K is derived from multi-key servers to resist brute-force attacks.

The comparative results obtained by simulating these three scenarios are presented in Fig. 3 to 7. These charts provide a clear and detailed comparison of the performance metrics, allowing us to deeply analyze the effectiveness and efficiency of our approach relative to other methods.

First, by changing the number of data blocks in the file from 1000 to 6000, with each block being the same size, we compare the time cost of the signature generation process. We set the data block size to a larger value for easier observation. As shown in Fig. 3, the time differences among the various schemes during the signature generation process are not significant, as most of them are identity-based signature schemes.

Next, we set the number of challenged data blocks to range from 300 to 1500, ensuring that the number of data blocks for each scheme remains the same and unchanged throughout this process. The comparison results are shown in Fig. 4. In the generation of auditing proof, the time taken by each scheme varies. Our scheme is divided into two phases during this process: the generation of data blocks and signature

Table 7
The comparison on the data recovery.

Scheme	Minimum Threshold	Key Mechanisms
Scheme [9]	—	Multi-Copy Recovery
Scheme [30]	$\leq 33\%$	RS Erasure Code
Our Scheme	$\leq 50\%$	MBR Regenerating Code

Table 8
The comparison of fault tolerance capabilities.

Scheme	10%	30%	50%
Scheme [9]	✓	✓	—
Scheme [30]	✓	✓	×
Our Scheme	✓	✓	✓

aggregation by the CSP_k , and the re-aggregation of proof collected from the CSP_k by the CO. Therefore, the time overhead is relatively high.

Finally, during the audit proof verification phase, as shown in Fig. 5, since our scheme generates the aggregation in advance on the CO, the verification process is relatively simple, resulting in lower time overhead.

We can observe from the comparison of the testing results that our suggested strategy is competitive when it comes to auditing proof verification and initial data upload. Nevertheless, we clarify that other techniques do not take into account data recovery and deduplication at the same time, whereas our protocol is the first to do so. Consequently, it is clear that our scheme works well and can be applied in real-world scenarios in the future.

6.6. Data recovery performance

The hardware configuration used in this part of the experiment is the same as that of Section 6.5. To evaluate the efficiency of our data recovery mechanism, we conduct experiments under various corruption ratios (10% to 50%) and file sizes (2GB to 10GB). The recovery time and bandwidth overhead are measured and compared with the scheme [9] and scheme [30].

In Tables 7 and 8, we compare the key data recovery technologies used in each scheme and analyze the minimum recoverable thresholds. From the comparison, we can see that the recoverability of scheme [9] depends on the number of replicas, while scheme [30] can only tolerate up to 33% corruption. In contrast, our scheme can still recover even when corruption reaches 50%, demonstrating stronger fault tolerance.

Subsequently, we primarily analyzed the data recovery scenarios when 30% of the data blocks are corrupted, under the condition that the file consists of 3000 blocks. In Fig. 6, we compared the recovery bandwidth for different data file sizes. The findings reveal that: Scheme [9] requires downloading the entire replica document during recovery, resulting in bandwidth consumption proportional to the file size. Our scheme achieves partial data recovery through matrix operations without requiring full file downloads.

Under the same conditions, we experimentally measured the time overhead of each scheme during data recovery in Fig. 7. It can be observed that the time varies with the file size. Overall, our scheme demonstrates superior performance in recovery efficiency.

6.7. Gas cost analysis

To quantify the real-world economic cost of our design, we benchmarked the Gas consumption of the core auditing function, **ProofVer**. The measurements were conducted on a local fork of the Ethereum mainnet using the industry-standard Foundry toolkit, ensuring the Gas costs accurately reflect live network conditions. Table 9 summarizes the estimated cost in USD for a single audit, based on a Gas price of 20 Gwei and an ETH price of \$2,500.

Table 9
The cost for a single audit.

Number of Challenged Blocks	Estimated Gas	Estimated Cost (USD)
30	1.83×10^5	\$9.15
60	3.06×10^5	\$15.30
90	4.29×10^5	\$21.45
120	5.52×10^5	\$27.60

The analysis confirms that on-chain verification incurs a tangible cost. However, when contextualized within the low-frequency, high-value audit scenario our scheme is designed for, this cost is not prohibitive. This cost-benefit trade-off justifies the use of a public blockchain for its superior trust properties in this specific context.

7. Conclusions

The cloud auditing system proposed in this paper successfully integrates cutting-edge technologies including multi-key server-assisted key generation, minimum bandwidth regeneration code, and blockchain-assisted transparent auditing. Through multi-key servers, the uniqueness of data file storage is ensured, while data block recovery efficiency is guaranteed by regeneration code. By leveraging blockchain, the scheme achieves a strong trust model, and our cost analysis confirms its feasibility for targeted, high-value audit scenarios. Performance analysis demonstrates that the system exhibits significant advantages in reducing communication redundancy, computational costs, and storage overheads. Therefore, the proposed system establishes a robust foundation for practical cloud auditing that is not only technologically superior but also economically viable. This solution provides an efficient and reliable approach for auditing data integrity and confidentiality in cloud storage environment.

CRedit authorship contribution statement

Peiru Yang: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Conceptualization; **Jinyong Chang**: Project administration, Methodology, Funding acquisition, Data curation; **Youtong Wang**: Supervision, Investigation, Formal analysis, Data curation; **Xia Zhou**: Project administration, Methodology, Investigation; **Yanyan Ji**: Writing – review & editing, Formal analysis.

Data availability

Data will be made available on request.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors would like to thank the anonymous referees for their invaluable suggestions and comments. This work is supported in part by the [National Natural Science Foundation of China](#) under Grant 62476212, in part by the Key Research and Development Program Project of Shaanxi Province under Grant 2025CY-YBXM-007, in part by the Natural Science Foundation Project of Shaanxi Province under Grant 2025JC-YBMS-774, and in part by Open Project of ISN National Key Laboratory under Grant ISN25-02.

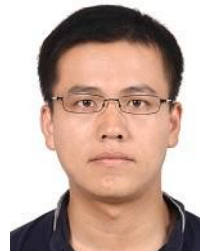
References

- [1] X. Li, S. Shang, S. Liu, et al, An identity-Based data integrity auditing scheme for cloud-Based maritime transportation systems, *IEEE Trans. Intell. Transp. Syst.* 24 (2) (2023) 2556–2567.

- [2] G. Nagarajan, K. Kumar, Security threats and challenges in public cloud storage, in: 2021 International Conference on Advance Computing and Innovative Technologies in Engineering, 2021, pp. 97–100.
- [3] J. Zhao, H. Huang, D. He, et al, IB-IADR: Enabling identity-Based integrity auditing and data recovery with fault localization for multicloud storage, *IEEE Internet Things J.* 11 (16) (2024) 27214–27231.
- [4] W. Shen, J. Qin, J. Yu, et al, Data integrity auditing without private key storage for secure cloud storage, *IEEE Trans. Cloud Comput.* 9 (4) (2021) 1408–1421.
- [5] Q. Zhang, S. Qian, J. Cui, et al, Blockchain-Based privacy-Preserving deduplication and integrity auditing in cloud storage, *IEEE Trans. Comput.* 74 (5) (2025) 1717–1729.
- [6] X. Tu, Wang, et al Du, An improved multi-Copy cloud data auditing scheme and its application, *J. King Saud Univ. Comput. Inf. Sci.* 35 (2023) 120–130.
- [7] H. Yuan, X. Chen, J. Wang, et al, Blockchain-Based public auditing and secure deduplication with fair arbitration, *Inf. Sci.* 541 (2020) 409–425.
- [8] A. Kumar, K. Chatterjee, An efficient blockchain-Based auditing scheme for EHR in intelligent healthcare system, in: 2023 IEEE 2Nd International Conference on Industrial Electronics: Developments and Applications, 2023, pp. 116–120.
- [9] J. Zhao, H. Huang, D. He, IB-IADR: Enabling identity-Based integrity auditing and data recovery with fault localization for multicloud storage, *IEEE Internet Things J.* 11 (16) (2024) 27214–27231.
- [10] M. Miao, J. Wang, H. Li, et al, Secure multi-Server-Aided data deduplication in cloud computing, *Pervasive Mob Comput* 24 (2015) 129–137.
- [11] W. Liang, Y. Fan, K. Li, et al, Secure data storage and recovery in industrial blockchain network environments, *IEEE Trans. Ind. Inf.* 16 (10) (2020) 6543–6552.
- [12] G. Ateniese, R. Burns, R. Curtmola, et al, Provable data possession at untrusted stores, in: Proceedings of the 14Th ACM Conference on Computer and Communications Security, the 14th ACM Conference on Computer and Communications Security, 2007, pp. 598–609.
- [13] A. Juels, B.S. Kaliski, PORs: Proofs of retrievability for large files, 2007, pp. 584–597.
- [14] Hovav, W. Shacham, Brent, Compact proofs of retrievability, *J. Cryptol.* 26 (3) (2013) 442–483.
- [15] X. Yang, X. Pei, M. Wang, et al, Multi-Replica and multi-Cloud data public audit scheme based on blockchain, *IEEE Access* 8 (2020) 144809–144822.
- [16] C. Wang, Y. Sun, B. Liu, et al, Blockchain-Based dynamic cloud data integrity auditing via non-Leaf node sampling of rank-Based merkle hash tree, *IEEE Trans. Network Sci. Eng.* 11 (5) (2024) 3931–3942.
- [17] Y. Zhang, C. Xu, X. Lin, et al, Blockchain-Based public integrity verification for cloud storage against procrastinating auditors, *IEEE Trans. Cloud Comput.* 9 (3) (2021) 923–937.
- [18] J.R. Douceur, A. Adya, W.J. Bolosky, Reclaiming space from duplicate files in a serverless distributed file system, in: Annual International Conference on the Theory and Applications of Cryptographic Techniques, 2002, pp. 617–624.
- [19] M. Bellare, S. Keelveedhi, T. Ristenpart, Message-Locked encryption and secure deduplication, in: Proc. EUROCRYPT, EUROCRYPT, 2013, pp. 296–312.
- [20] B. Mihir, K. Sriram, R. Thomas, DupLESS: server-Aided encryption for deduplicated storage, in: 22Nd USENIX Security Symposium, 2013, pp. 179–194.
- [21] S. Halevi, D. Harnik, B. Pinkas, et al, Proofs of ownership in remote storage systems, in: Proceedings of the 18Th ACM Conference on Computer and Communications Security, the 18th ACM conference on Computer and communications security, 2011, pp. 491–500.
- [22] T.Y. Youn, K.Y. Chang, Bi-Birectional and concurrent proof of ownership for stronger storage services with deduplication, *Science China Inf. Sci.* 61 (2018) 1–11.
- [23] J. Yuan, S. Yu, Secure and constant cost public cloud storage auditing with deduplication, in: 2013 IEEE Conference on Communications and Network Security, 2013, pp. 145–153.
- [24] J. Li, J. Li, D. Xie, et al, Secure auditing and deduplicating data in cloud, *IEEE Trans. Comput.* 65 (8) (2016) 2386–2396.
- [25] Y. Miao, K. Gai, L. Zhu, et al, Blockchain-Based shared data integrity auditing and deduplication, *IEEE Trans Dependable Secure Comput* 21 (4) (2024) 3688–3703.
- [26] X. Zheng, W. Shen, Y. Su, et al, DIADD: Secure deduplication and efficient data integrity auditing with data dynamics for cloud storage, *IEEE Trans. Netw. Serv. Manage.* 22 (1) (2025) 299–316.
- [27] M. Song, Z. Hua, Y. Zheng, et al, Blockchain-Based deduplication and integrity auditing over encrypted cloud storage, *IEEE Trans. Dependable Secure Comput.* 20 (6) (2023) 4928–4945.
- [28] A.G. Dimakis, P.B. Godfrey, Y. Wu, et al, Network coding for distributed storage systems, *IEEE Trans. Inf. Theory* 56 (9) (2010) 4539–4551.
- [29] J. Li, H. Yan, Y. Zhang, Efficient identity-Based provable multi-Copy data possession in multi-Cloud storage, *IEEE Trans. Cloud Comput.* 10 (1) (2022) 356–365.
- [30] L. Qiao, Y. Li, Y. Ding, Lightweight and decentralized cross-Cloud auditing with data recovery, *IEEE Trans. Serv. Comput.* 17 (1) (2024) 60–73.



Peiru Yang received the BS in computer science and technology from Xi'an University of Architecture and Technology (XAUAT) in 2023. Now, she is a Master candidate in School of Information and Control Engineering, XAUAT, 710055, Xi'an, Shaanxi, China. Her research interests include information security and cryptography.



Jinyong Chang received the BS and MS degrees in mathematics from Shanxi University and Capital Normal University in 2004 and 2009, respectively, and the Ph.D. Degree in Information Security from the SKLOIS, Institute of Information Engineering, Chinese Academy of Sciences, in 2015. He was a Post Doc. in Peking University, Beijing, China. Now, he is a professor in School of Information and Control, XAUAT. His research interests mainly focus on cryptography and information security.



Youtong Wang was born in 2000, and now is a Master candidate in School of Information and Control Engineering, Xi'an University of Architecture and Technology (XAUAT), 710055, Xi'an, Shaanxi, China. His research interests include information security and cryptography.



Xia Zhou was born in 2000, and now is a Master candidate in School of Information and Control Engineering, Xi'an University of Architecture and Technology (XAUAT), 710055, Xi'an, Shaanxi, China. Her research interests include information security and cryptography.



Yanyan Ji received the B S and M S degrees in school of electrical engineering and automation from Henan Polytechnic University in 2005, and school of information and communication engineering from North University of China in 2018. Now, she is a lecturer in School of Information and Control Engineering, Xi'an University of Architecture and Technology (XAUAT), Shaanxi, China. Her research interests mainly focus on communication and information security.